Table des matières

Cours 04 — Fonctions de fenêtrage 1
Objectifs 1
Théorie 1
Points clés à retenir 10
Exercices 11

Cours 04 — Fonctions de fenêtrage

Module 4/12 — Formation SQL avancé

Comment lire ce module — C'est le cours qui sépare le SQL de débutant du SQL de pro, celui que les recruteurs testent le plus. On part d'un problème très concret que le `GROUP BY` ne sait pas résoudre, puis on découvre l'outil qui le règle, la fonction de fenêtrage. Une idée à la fois, chaque fonction illustrée sur l'entrepôt AdventureWorks DW. La théorie est ici ; les deux laboratoires (`Labo_04_OLAP_SQL.md` et `Labo_04_OLTP_SQL.md`) vous font tout taper.

Objectifs

À la fin de ce module, vous saurez :

- expliquer la différence entre un `GROUP BY`, qui écrase les lignes, et une fenêtre, qui les garde ;
- lire et écrire la clause `OVER (PARTITION BY … ORDER BY … )` ;
- classer des lignes avec `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, et choisir la bonne ;
- regarder la ligne précédente ou suivante avec `LAG` et `LEAD`, pour un calcul d'évolution ;
- calculer des cumuls, des moyennes mobiles et des parts du total ;
- maîtriser le cadre (`ROWS` contre `RANGE`) et éviter le piège du cadre par défaut ;
- filtrer sur le résultat d'une fenêtre avec `QUALIFY`, et sans lui quand le moteur ne l'a pas.

Théorie

1. Le problème : calculer sans écraser les lignes

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, partie 1.

Vous savez déjà agréger. « Le total vendu par produit », c'est un `GROUP BY` :

```
SELECT ProductKey, SUM(SalesAmount) AS total_produit
FROM dbo.FactInternetSales
GROUP BY ProductKey;
```

Le `GROUP BY` fusionne toutes les ventes d'un produit en une seule ligne. C'est parfait pour un résumé, mais vous perdez le détail. Impossible, dans ce résultat, de voir une vente individuelle à côté du total de son produit.

Or c'est une question qui revient sans arrêt en analytique :

- afficher chaque vente avec, en regard, le total du produit ou sa part du total ;
- classer les produits du plus cher au moins cher sans les regrouper ;
- comparer le chiffre d'un mois à celui du mois précédent.

La **fonction de fenêtrage** fait exactement ça : elle calcule une valeur qui regarde plusieurs lignes, mais sans en supprimer aucune. Chaque ligne reste ; on lui ajoute une colonne calculée.

L'image à retenir — Le `GROUP BY` réduit un paquet de lignes à une seule. La fenêtre glisse sur les lignes et, pour chacune, jette un œil autour d'elle, sans rien fusionner.

2. La syntaxe : `OVER (…)`

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, partie 1.

Une fonction de fenêtrage, c'est une fonction habituelle (`SUM`, `COUNT`, un classement) suivie du mot-clé `OVER`, qui définit la fenêtre, c'est-à-dire les lignes que la fonction a le droit de regarder. Trois ingrédients, tous optionnels :

Ingrédient	Ce qu'il fait	Ce qu'il vaut si on l'omet
`PARTITION BY`	découpe les lignes en groupes, comme un `GROUP BY` mais sans fusionner. La fonction se calcule séparément dans chaque groupe	la fenêtre couvre toute la table
`ORDER BY` (à l'intérieur du `OVER`)	trie les lignes dans la fenêtre. Indispensable dès qu'un ordre a un sens : classer, cumuler, regarder la ligne précédente	aucun ordre, donc aucun cumul possible

© 2026 iAlexMG. Tous droits réservés.

Ingrédient	Ce qu'il fait	Ce qu'il vaut si on l'omet
le cadre (`ROWS` ou `RANGE`)	restreint encore la fenêtre à une portion autour de la ligne courante, les trois précédentes par exemple. Voir la section 6	un cadre par défaut s'applique, et c'est un piège (section 6)

L'écriture complète se lit donc de gauche à droite : `fonction() OVER (PARTITION BY … ORDER BY … cadre)`, soit « découpe, puis trie, puis restreins ».

Le total par produit en gardant chaque vente :

```sql
SELECT SalesOrderNumber, ProductKey, SalesAmount,
       SUM(SalesAmount) OVER (PARTITION BY ProductKey) AS total_produit
FROM dbo.FactInternetSales;
```

Chaque vente ressort, plus une colonne `total_produit` répétée pour toutes les ventes du même produit. Comparez avec le `GROUP BY` de la section 1 : même calcul, mais ici rien n'est perdu.

Attention à l'ordre d'exécution (rappel du Cours 01) — Les fonctions de fenêtrage se calculent à l'étape du `SELECT`, après le `WHERE` et le `GROUP BY`. Conséquence : on ne peut pas les utiliser dans un `WHERE`. Pour filtrer sur une fenêtre, il faudra le `QUALIFY` (section 7) ou une sous-requête.

3. Classer les lignes : `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, sous-partie 2.1.

Quatre fonctions attribuent un numéro à chaque ligne selon l'ordre choisi. Elles n'ont pas besoin d'argument, juste d'un `ORDER BY` dans le `OVER`.

Classons les produits du plus cher au moins cher :

```sql
SELECT EnglishProductName, ListPrice,
       ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS no_ligne,
       RANK()       OVER (ORDER BY ListPrice DESC) AS rang,
       DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS rang_dense
FROM dbo.DimProduct
WHERE ListPrice > 0;
```

Ce qui les distingue, c'est leur façon de gérer les égalités, deux produits au même prix :

Fonction	À quoi elle sert	En cas d'égalité
`ROW_NUMBER`	numéroter sans doublon (1, 2, 3, 4)	départage arbitrairement, deux numéros différents
`RANK`	classer avec rangs égaux, puis sauter	1, 2, 2, 4 (le 3 saute)

© 2026 iAlexMG. Tous droits réservés.

Fonction	À quoi elle sert	En cas d'égalité
`DENSE_RANK`	classer avec rangs égaux, sans sauter	1, 2, 2, 3
`NTILE(n)`	répartir en n paquets de taille aussi égale que possible	quartiles, déciles

Notre entrepôt donne l'exemple tout fait, et il est plus parlant qu'un tableau de principe. Cinq produits partagent le prix le plus haut du catalogue, les Road-150 Red en tailles 44, 48, 52, 56 et 62, à 3 578,27 $. Voici ce que les trois fonctions en font, et ce qu'elles font du produit suivant, le Mountain-100 Silver à 3 399,99 $:

Produit	ROW_NUMBER	RANK	DENSE_RANK
Road-150 Red, les cinq tailles	1, 2, 3, 4, 5 dans un ordre quelconque	1 pour les cinq	1 pour les cinq
Mountain-100 Silver, le suivant	6	6	2

Tout est là. `ROW_NUMBER` a dû choisir un ordre entre cinq produits que rien ne départage, et ce choix peut changer d'une exécution à l'autre. `RANK` leur donne le même rang, puis saute les cinq places consommées et reprend à 6. `DENSE_RANK` leur donne le même rang et enchaîne à 2, parce qu'il compte les prix distincts et non les lignes. Trois réponses justes à trois questions différentes : « quelle est cette ligne », « combien de produits sont plus chers », « combien de prix sont plus élevés ».

Le cas d'usage roi, c'est le « top N par groupe ». Avec `PARTITION BY`, on classe à l'intérieur de chaque groupe. Le produit le plus cher de chaque sous-catégorie :

```sql
SELECT ProductSubcategoryKey, EnglishProductName, ListPrice,
       ROW_NUMBER() OVER (PARTITION BY ProductSubcategoryKey
                          ORDER BY ListPrice DESC) AS rang_dans_sous_cat
FROM dbo.DimProduct
WHERE ListPrice > 0;
```

On obtient un classement qui repart à 1 dans chaque sous-catégorie. Reste à ne garder que `rang_dans_sous_cat = 1`, et c'est le rôle du `QUALIFY` (section 7).

`NTILE(4)` découpe la population en quatre paquets aussi égaux que possible, ce qui est pratique pour dire qu'un produit est dans le quart le plus cher :

```sql
SELECT EnglishProductName, ListPrice,
       NTILE(4) OVER (ORDER BY ListPrice DESC) AS quartile_prix
FROM dbo.DimProduct
WHERE ListPrice > 0;
```

« Aussi égaux que possible » n'est pas une précaution de langage. Le catalogue compte 395 produits tarifés, et 395 ne se divise pas par quatre : les paquets rendus sont de 99, 99, 99 et 98 lignes, les premiers recevant la ligne en trop. Plus instructif encore, la frontière entre le premier et le deuxième quartile tombe à 1 204,3248 $, un prix que portent trois produits : un se retrouve dans le premier quartile, les deux autres dans le deuxième. `NTILE` répartit des lignes, pas des valeurs, et il coupe où le compte l'oblige à couper, quitte à séparer des ex aequo. Retenez-en qu'un quartile de `NTILE` désigne un rang, jamais un seuil de prix.

© 2026 iAlexMG. Tous droits réservés.

4. Regarder la ligne d'à côté : LAG et LEAD

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, sous-partie 2.3.

`LAG` va chercher la valeur d'une ligne précédente, `LEAD` d'une ligne suivante, selon l'`ORDER BY` de la fenêtre. C'est l'outil des évolutions : combien ce mois-ci par rapport au mois dernier.

Prenons les ventes par mois. On calcule d'abord le total mensuel, par une sous-requête, puis on applique la fenêtre par-dessus :

```sql
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois)          AS ventes_mois_prec,
       ventes - LAG(ventes) OVER (ORDER BY annee, mois) AS variation
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
) AS mensuel
ORDER BY annee, mois;
```

Pour chaque mois, `LAG(ventes)` renvoie le total de la ligne précédente ; la soustraction donne la variation. La ligne précédente est ici le mois précédent parce qu'aucun mois ne manque : les 38 mois de décembre 2010 à janvier 2014 ont tous des ventes, mesuré. `LAG` ne regarde pas le calendrier, il regarde l'ordre des lignes, et un mois sans vente n'a pas de ligne. Découpées par région et par sous-catégorie, 32 des 149 séries mensuelles de l'entrepôt ont au moins un trou : la ligne de novembre 2013 des vélos de montagne du Nord-Est s'y compare à novembre 2012, douze mois plus tôt, sous le nom de « mois précédent ». La parade est de fabriquer d'abord le calendrier complet, puis d'y raccrocher les ventes (Cours 05, section 6).

La toute première ligne n'a pas de précédent : `LAG` y vaut `NULL`, et on peut fournir une valeur par défaut avec `LAG(ventes, 1, 0)`. Sur une variation, ce zéro est un mensonge : la première ligne affiche alors tout son total comme une hausse, 43 421,04 $ en décembre 2010, là où `NULL` disait qu'il n'y a rien à comparer.

Ça devient lourd ? Répéter `LAG(ventes) OVER (…)` alourdit la requête, et la sous-requête imbriquée n'aide pas à la lecture. Le Cours 05 nettoiera tout ça avec `WITH`.

Portabilité — `LAG`, `LEAD`, `ROW_NUMBER` et `RANK` sont standard : identiques sur nos deux terrains, DuckDB et PostgreSQL, où ce cours les a tous lancés, et sur SQL Server, Oracle et Snowflake d'après leur documentation. Les arguments optionnels aussi : `LAG(ventes, 1, 0)` rend 0 sur la première ligne au lieu de `NULL`, vérifié des deux côtés.

Trois fonctions voisines, en revanche, ne voyagent pas aussi bien, et c'est le piège classique de la famille. `FIRST_VALUE`, `LAST_VALUE` et `NTH_VALUE` dépendent du cadre, donc du défaut décrit à la section 6. Nos deux terrains appliquent le défaut standard : `LAST_VALUE(x) OVER (ORDER BY d)` s'arrête à la ligne courante et rend donc la valeur de la ligne courante, ce qui n'est jamais ce qu'on voulait. Mesuré des deux côtés sur trois lignes : 10, 20, 30, là où le cadre explicite `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` rend bien 30 partout. Snowflake, lui, leur donne d'après sa documentation un cadre par défaut

© 2026 iAlexMG. Tous droits réservés.

qui va jusqu'au bout de la partition, et précise que ce comportement n'est pas conforme à la norme. La même requête y rendrait donc 30, 30, 30. Écrivez toujours le cadre en entier avec ces trois fonctions : c'est la seule écriture qui dit ce que vous voulez et qui le dit partout pareil.

5. Cumuler, lisser, rapporter au total

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, sous-partie 2.4.

Les fonctions d'agrégat classiques (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`) deviennent des fonctions de fenêtrage dès qu'on leur ajoute `OVER`. Trois usages incontournables.

Rappel — `MIN`, `MAX` et les autres agrégats, avec ce qu'ils font des `NULL`, sont présentés au Cours 02, section 7.

Le cumul (*running total*). Un `ORDER BY` dans le `OVER` transforme le `SUM` en somme qui s'accumule ligne après ligne :

```
SELECT annee, mois, ventes,
       SUM(ventes) OVER (PARTITION BY annee ORDER BY mois)  AS cumul_annuel,
       SUM(ventes) OVER (ORDER BY annee, mois)              AS cumul_total
FROM (
    SELECT EXTRACT(YEAR  FROM h.orderdate)::int AS annee,
           EXTRACT(MONTH FROM h.orderdate)::int AS mois,
           SUM(d.linetotal)                     AS ventes
    FROM sales.salesorderheader AS h
    JOIN sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
    GROUP BY 1, 2
) AS mensuel
ORDER BY annee, mois;
```

Les deux colonnes disent la différence que fait un `PARTITION BY`. `cumul_total` additionne le mois courant et tous les mois précédents, sans fin. `cumul_annuel` repart à zéro à chaque changement d'année, parce que la partition redécoupe la fenêtre. Le passage d'une année à l'autre le montre :

annee	mois	ventes	cumul_annuel	cumul_total
2022	12	2 504 196,21	14 561 051,59	14 561 051,59
2023	1	2 076 791,61	2 076 791,61	16 637 843,20

Un cumul qu'on appelle « annuel » sans partitionner est l'erreur la plus courante du fenêtrage, et la plus difficile à voir : la colonne a l'air juste tant qu'on ne regarde pas le mois de janvier.

La part du total (*percent of total*). Une fenêtre vide, `OVER ()`, couvre toute la table, ce qui est parfait pour un dénominateur global.

```
SELECT EnglishProductName, ListPrice,
       ListPrice / SUM(ListPrice) OVER () AS part_du_total
FROM dbo.DimProduct
WHERE ListPrice > 0;
```


© 2026 iAlexMG. Tous droits réservés.

La moyenne mobile. Avec un cadre (section 6), on lisse sur une fenêtre glissante, ici la moyenne des trois derniers mois :

```
AVG(ventes) OVER (ORDER BY annee, mois ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```

6. Le cadre : ROWS contre RANGE (et le piège du défaut)

Labo : `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, sous-partie 2.4.

Le cadre précise, à l'intérieur de la fenêtre triée, quelles lignes entrent dans le calcul par rapport à la ligne courante. On l'écrit avec `ROWS BETWEEN … AND …` :

Cadre	Lignes prises en compte
`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	du début jusqu'à la ligne courante, donc un cumul
`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`	la ligne courante et les deux d'avant, donc une moyenne mobile sur trois
`ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING`	de la ligne courante jusqu'à la fin

`ROWS` compte des lignes physiques, `RANGE` regarde les valeurs. `ROWS 2 PRECEDING` désigne les deux lignes juste avant ; `RANGE` regroupe les lignes de même valeur d'`ORDER BY`, ce qui est utile mais souvent source de surprises. Règle simple : pour un cumul ou une moyenne mobile, écrivez `ROWS`, c'est prévisible.

Prévisible à une condition, qu'il faut énoncer parce qu'elle est presque toujours tue : `ROWS` ne l'est que si l'`ORDER BY` de la fenêtre ordonne les lignes **de façon unique**. Sinon, l'ordre entre ex aequo est arbitraire, et le résultat peut changer d'une exécution à l'autre sans qu'une donnée ait bougé. La documentation de PostgreSQL le dit sans détour : le mode `ROWS` peut produire des résultats imprévisibles si l'`ORDER BY` n'ordonne pas les lignes uniquement.

Trois lignes portant le même jour suffisent à le voir. Une moyenne mobile en `ROWS BETWEEN 1 PRECEDING AND CURRENT ROW`, ordonnée par le seul jour, additionne « la ligne d'avant », mais laquelle des trois ? Le moteur en choisit une, et rien ne garantit que ce soit la même demain.

La parade se choisit selon la question. Si le grain est déjà le bon, ajoutez au `ORDER BY` une colonne qui départage, une clé technique par exemple, et l'ordre redevient unique. Si vous vouliez en réalité raisonner par jour et non par ligne, alors ce n'était pas un cadre en lignes qu'il vous fallait : agrégez d'abord au grain voulu, puis posez la fenêtre sur le résultat. Une moyenne mobile sur douze mois écrite `ROWS BETWEEN 11 PRECEDING` compte douze **lignes**, pas douze mois, et si votre table porte plusieurs lignes par mois, elle ne calcule pas ce que son nom annonce.

Le piège du cadre par défaut — Dès qu'il y a un `ORDER BY` dans le `OVER` sans cadre explicite, le moteur applique `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Le plus souvent ça tombe bien, pour un cumul, mais s'il y a des valeurs d'ordre en double, le `RANGE` les additionne toutes d'un coup, et le résultat surprend. En cas de doute, précisez `ROWS`.

Quatre lignes suffisent à le voir, et l'essai vaut mieux que la description. On se fabrique une petite table où deux lignes partagent la même valeur d'ordre :

```
CREATE OR REPLACE TEMP TABLE cadre (jour INTEGER, montant INTEGER);
INSERT INTO cadre VALUES (1, 10), (2, 20), (2, 30), (3, 40);
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT jour, montant,
       SUM(montant) OVER (ORDER BY jour) AS defaut,
       SUM(montant) OVER (ORDER BY jour
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS avec_rows
FROM cadre
ORDER BY jour, montant;
```

jour	montant	defaut	avec_rows
1	10	10	10
2	20	60	30
2	30	60	60
3	40	100	100

Regardez la deuxième ligne. Le cumul explicite en `ROWS` vaut 30, ce qu'on attend d'un cumul arrêté à la ligne courante. Le cumul par défaut vaut 60 : il a déjà avalé la ligne du dessous, qui porte le même `jour` et qui est donc sa jumelle au sens du `RANGE`. Les deux lignes du jour 2 reçoivent la même valeur, et la colonne cesse de progresser ligne à ligne. Ce comportement est le même sur nos deux terrains, mesuré sur DuckDB et sur PostgreSQL, et il n'a rien d'un défaut : c'est la norme. Il ne devient un piège que parce qu'on ne l'a pas demandé.

7. Filtrer sur une fenêtre : `QUALIFY`

Labo : `Labo_04_OLAP_SQL.md` et `Labo_04_OLTP_SQL.md`, sous-partie 2.2.

Rappelez-vous qu'une fonction de fenêtrage se calcule après le `WHERE` : on ne peut donc pas l'y utiliser. `QUALIFY` règle ça. Il est au fenêtrage ce que le `HAVING` est au `GROUP BY`, un filtre qui s'applique après le calcul de la fenêtre, et il tourne sur l'entrepôt de cette formation.

Le fameux « top N par groupe » devient limpide. Les trois produits les plus chers de chaque sous-catégorie :

```sql
SELECT ProductSubcategoryKey, EnglishProductName, ListPrice
FROM dbo.DimProduct
WHERE ListPrice > 0
QUALIFY ROW_NUMBER() OVER (PARTITION BY ProductSubcategoryKey
                           ORDER BY ListPrice DESC) <= 3;
```

Une seule requête, pas de sous-requête. Sur l'entrepôt, elle rend 92 lignes et non 111 : les 37 sous-catégories qui ont des produits tarifés n'en ont pas toutes trois, et onze d'entre elles rendent donc moins de trois lignes. `QUALIFY` ne complète rien, il filtre ce qui existe. C'est l'un des raccourcis qui rendent le SQL analytique agréable, et c'est une question d'entretien classique.

© 2026 iAlexMG. Tous droits réservés.

Portabilité — `QUALIFY` n'est pas dans la norme, et c'est là qu'il faut être précis, parce que la répartition ne recoupe pas celle qu'on imagine. Il tourne sur DuckDB, donc sur la voie OLAP de cette formation, et sur Snowflake d'après sa documentation ; PostgreSQL ne le connaît pas et répond « erreur de syntaxe sur ou près de QUALIFY ». Ce n'est donc pas une facilité de grande plateforme opposée aux moteurs modestes : c'est une facilité analytique, que les moteurs analytiques ont adoptée et que les transactionnels ignorent. Sur PostgreSQL, on enveloppe la fenêtre dans une sous-requête (ou une CTE, Cours 05) et on filtre à l'extérieur :

```sql
SELECT * FROM (
    SELECT productsubcategoryid, name, listprice,
           ROW_NUMBER() OVER (PARTITION BY productsubcategoryid
                              ORDER BY listprice DESC) AS rang
    FROM production.product
    WHERE listprice > 0
) t
WHERE rang <= 3;
```

Les noms sont ceux du transactionnel : `production.product` n'a ni `ProductSubcategoryKey`, ni `EnglishProductName`, ni `ListPrice`. Ce sont ceux de `dbo.DimProduct`, dans l'entrepôt, et la requête de l'encadré échouerait avec eux.

8. Sur les deux terrains : OLTP et OLAP

Labo : `Labo_04_OLTP_SQL.md`, partie 3.

Les fonctions de fenêtrage sont du SQL standard : elles se comportent à l'identique sur un moteur transactionnel et sur un entrepôt. Ce qui change d'un monde à l'autre, ce n'est pas la fonction, c'est le chemin pour arriver aux lignes sur lesquelles vous la posez.

Ce qu'on veut	Voie OLTP (PostgreSQL, schéma normalisé)	Voie OLAP (DuckDB, étoile)
Le montant d'une vente	jointure `salesorderheader` et `salesorderdetail`	tout est dans `FactInternetSales`
L'année et le mois	`EXTRACT` sur `orderdate`	colonnes de `DimDate`, déjà prêtes
La sous-catégorie d'un produit	jointure vers `productsubcategory`	colonne de `DimProduct`, repliée
Filtrer sur une fenêtre	sous-requête puis `WHERE`	`QUALIFY`, en une ligne
La syntaxe de la fenêtre	`OVER (PARTITION BY … ORDER BY …)`	identique, au caractère près

Rappel — `EXTRACT`, qui sort l'année ou le mois d'une date, est présenté au Cours 02, section 6.

Trois conséquences pratiques. Sur le transactionnel, vos requêtes analytiques sont plus longues, non parce que le calcul est plus dur, mais parce qu'il faut reconstituer le contexte à chaque fois. Sur l'entrepôt, le contexte est déjà là, ce qui est précisément la raison d'être de la modélisation dimensionnelle. Et le passage d'un monde à l'autre ne demande jamais de réapprendre les fenêtres : il demande de relire le schéma.

© 2026 iAlexMG. Tous droits réservés.

Une quatrième, moins évidente, se voit sur la performance. Une fenêtre exige un tri : le moteur doit ordonner les lignes de chaque partition avant de pouvoir numéroter, cumuler ou regarder la ligne d'à côté. La question est donc de savoir ce qui peut lui épargner ce tri, et la réponse n'est pas la même des deux côtés.

Sur PostgreSQL, un index qui porte déjà l'ordre de la fenêtre le lui épargne pour de bon. Mesuré sur les 121 317 lignes de `sales.salesorderdetail`, avec une fenêtre partitionnée par `productid` et ordonnée par `linetotal` : sans index, le plan montre un `Sort` que le moteur déverse sur disque, « external merge Disk: 3112kB » ; avec un index sur ces deux colonnes dans cet ordre, le `Sort` disparaît du plan, remplacé par un `Index Scan` qui livre les lignes déjà rangées. La même mesure sur les 504 lignes de `production.product` ne montre aucune différence : la table est trop petite pour que l'index vaille le détour, et le planificateur l'ignore. Un index n'aide donc pas parce qu'il existe, mais parce qu'il évite un travail qui coûtait quelque chose.

Sur DuckDB, le même essai ne donne rien, et c'est intéressant. Un index créé sur les colonnes de partition et d'ordre ne change ni le plan, toujours un balayage suivi de l'opérateur `WINDOW`, ni le temps, 15 ms contre 15 ms sur `FactInternetSales`. L'index d'un moteur en colonnes ne sert pas à livrer un ordre, il sert à retrouver des lignes ; le tri d'une fenêtre, lui, se fait en mémoire sur des colonnes déjà compactes, et c'est de là que vient la vitesse. Le levier, côté entrepôt, n'est donc pas l'index : c'est de ne lire que les colonnes utiles et de ne pas partitionner sur ce dont on n'a pas besoin.

Portabilité — Sur Snowflake, une table ordinaire n'a pas d'index, d'après sa documentation ; ses tables hybrides en ont. Son découpage en micro-partitions sert, toujours d'après sa documentation, à écarter des blocs de données devant un filtre, le mécanisme que le Cours 10 mesure sur l'entrepôt. Rien de ce qu'on a lu ne le présente comme un moyen d'épargner le tri d'une fenêtre : n'en attendez pas ce service sans l'avoir mesuré.

Le principe commun tient en une phrase : donner au moteur un moyen de ne pas tout trier. Ce qui change, c'est le moyen, et un réflexe rapporté d'un monde à l'autre y est souvent sans effet. Les plans d'exécution et les index sont le sujet du Cours 10.

Le même travail, deux fois — Le Cours 04 a deux laboratoires, `Labo_04_OLTP_SQL.md` et `Labo_04_OLAP_SQL.md`, qui posent les mêmes questions sur les deux schémas. Faites-les tous les deux et gardez-les ouverts côte à côte. C'est l'exercice de comparaison qui installe durablement la distinction entre le langage, qui ne change pas, et le terrain, qui change tout le temps.

Points clés à retenir

- `GROUP BY` écrase, la fenêtre garde. Une fonction de fenêtrage calcule sur plusieurs lignes sans en supprimer aucune : elle ajoute une colonne.
- `OVER (PARTITION BY … ORDER BY … cadre)` : `PARTITION BY` découpe sans fusionner, `ORDER BY` trie dans la fenêtre, le cadre restreint à une portion. Les trois sont optionnels, et `OVER ()` couvre toute la table.
- Classement : `ROW_NUMBER` sans doublon, `RANK` avec égalités et sauts, `DENSE_RANK` avec égalités sans saut, `NTILE(n)` en n paquets aussi égaux que possible, qui disent un rang et non un seuil. Avec `PARTITION BY`, on obtient le top N par groupe.
- `LAG` et `LEAD` regardent la ligne précédente ou suivante, d'où le calcul d'évolution d'un mois à l'autre. `NULL` au bord, sauf valeur par défaut fournie.
- Agrégats fenêtrés : `SUM(...) OVER (ORDER BY ...)` donne un cumul, `... OVER ()` donne la part du total, et l'ajout d'un cadre donne une moyenne mobile.

© 2026 iAlexMG. Tous droits réservés.

- Cadre : `ROWS` compte des lignes, ce qui est prévisible, `RANGE` regarde les valeurs. Pour un cumul ou une moyenne mobile, écrivez `ROWS`. Le cadre `RANGE` par défaut, quand il y a un `ORDER BY` sans cadre, donne la même valeur à toutes les lignes qui partagent la valeur d'ordre.
- `QUALIFY` filtre sur une fenêtre, comme `HAVING` sur un `GROUP BY`. Il tourne sur DuckDB, donc sur la voie OLAP ; PostgreSQL ne l'a pas, et on y passe par une sous-requête ou une CTE.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_04_SQL.md`. Faites-les après les deux laboratoires : plusieurs énoncés supposent que les manipulations ont été faites des deux côtés.

© 2026 iAlexMG. Tous droits réservés.